

MYSQL MINI PROJECT – RETAIL STORE

DONE BY – KHUSHALI KUMAWAT

QUESTIONS:

Level 1 Basics:

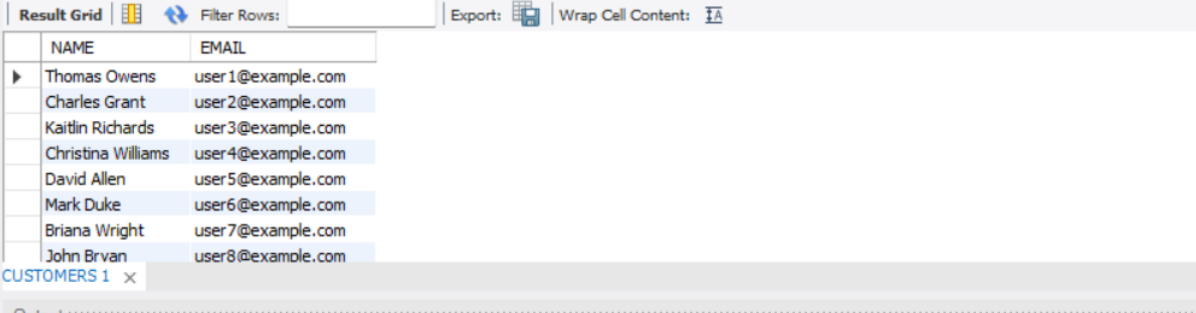
-- Construct SQL queries to answer the given business questions

1. Retrieve customer names and emails for email marketing

SQL QUERY:

SELECT NAME,EMAIL FROM CUSTOMERS;

```
2218 -- 1. Retrieve customer names and emails for email marketing
2219 -- This helps the marketing team extract basic customer contact details for campaigns
2220
2221 • SELECT NAME,EMAIL FROM CUSTOMERS;
2222
```



NAME	EMAIL
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Brvan	user8@example.com

-- 2. View complete product catalog with all available details.

SELECT * FROM PRODUCTS;

```

2223 -- 2. View complete product catalog with all available details
2224 -- The product manager may want to review all product listings in one go.
2225
2226 • SELECT * FROM PRODUCTS;
2227

```

Result Grid						
Filter Rows:		Edit:		Export/Import:		Wrap Cell Content: IA
PRODUCT_ID	NAME	CATEGORY	PRICE	STOCK_QUANTITY	ADDED_ON	
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50	
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46	
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03	
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37	
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14	
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08	
8	Everv Amount	Home	4173.60	156	2025-04-30 03:11:10	

PRODUCTS 2 x

-- 3. List all unique product categories

```
SELECT DISTINCT CATEGORY FROM PRODUCTS;
```

```

2228 -- 3. List all unique product categories
2229 -- Useful for analyzing the range of departments or for creating filters on the website.
2230 • SELECT DISTINCT CATEGORY FROM PRODUCTS;
2231

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

IA

	CATEGORY
▶	Home
	Clothing
	Electronics
	Toys
	Books

-- 4. Show all products priced above ₹1,000

```
SELECT * FROM PRODUCTS WHERE PRICE>1000;
```

```

2232 -- 4. Show all products priced above ₹1,000
2233 -- This helps identify high-value items for premium promotions or pricing strategy reviews.
2234
2235 • SELECT * FROM PRODUCTS WHERE PRICE>1000;
2236

```

Result Grid						
Filter Rows:		Edit:		Export/Import:		Wrap Cell Content: IA
PRODUCT_ID	NAME	CATEGORY	PRICE	STOCK_QUANTITY	ADDED_ON	
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50	
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46	
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03	
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37	
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08	
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10	
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57	
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15	

PRODUCTS 4 x

-- 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)

```
SELECT * FROM PRODUCTS WHERE PRICE BETWEEN 2000 AND 5000;
```

```
2237 -- 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)
2238 -- A merchandising team might need this to create a mid-tier pricing campaign.
2239
2240 • SELECT * FROM PRODUCTS WHERE PRICE BETWEEN 2000 AND 5000;
2241
```

PRODUCT_ID	NAME	CATEGORY	PRICE	STOCK_QUANTITY	ADDED_ON
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06

-- 6. Fetch data for specific customer IDs (e.g., from loyalty program list)

```
SELECT * FROM CUSTOMERS;
```

```
SELECT * FROM CUSTOMERS WHERE CUSTOMER_ID IN (5,6,7);
```

```
2241
2242 -- 6. Fetch data for specific customer IDs (e.g., from loyalty program list)
2243 -- This is used when customer IDs are pre-selected from another system
2244
2245 • SELECT * FROM CUSTOMERS;
2246 • SELECT * FROM CUSTOMERS WHERE CUSTOMER_ID IN(5,6,7);
2247
```

CUSTOMER_ID	NAME	EMAIL	PHONE	CREATED_AT
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
*	NULL	NULL	NULL	NULL

- 7. Identify customers whose names start with the letter 'A'

```
SELECT * FROM CUSTOMERS WHERE NAME LIKE 'A%';
```

```

2248 -- 7. Identify customers whose names start with the letter 'A'
2249 -- Used for alphabetical segmentation in outreach or app display.
2250
2251 • SELECT * FROM CUSTOMERS WHERE NAME LIKE 'A%';
2252

```

	CUSTOMER_ID	NAME	EMAIL	PHONE	CREATED_AT
▶	15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
	16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
	19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
	27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
•	NULL	NULL	NULL	NULL	NULL

-- 8. List electronics products priced under ₹3,000

```
SELECT * FROM PRODUCTS;
```

```
SELECT * FROM PRODUCTS WHERE CATEGORY='ELECTRONICS' AND PRICE < 3000;
```

```

2253 -- 8. List electronics products priced under ₹3,000
2254 -- Used by merchandising or frontend teams to showcase budget electronics.
2255 • SELECT * FROM PRODUCTS;
2256 • SELECT * FROM PRODUCTS WHERE CATEGORY='ELECTRONICS' AND PRICE < 3000;
2257

```

	PRODUCT_ID	NAME	CATEGORY	PRICE	STOCK_QUANTITY	ADDED_ON
▶	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
	44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
	47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
•	NULL	NULL	NULL	NULL	NULL	NULL

-- 9. Display product names and prices in descending order of price

```
SELECT NAME,PRICE FROM PRODUCTS ORDER BY PRICE DESC;
```

```

2258 -- 9. Display product names and prices in descending order of price
2259 -- This helps teams easily view and compare top-priced items.
2260
2261 • SELECT NAME,PRICE FROM PRODUCTS ORDER BY PRICE DESC;
2262

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	NAME	PRICE
▶	Response Indeed	4897.36
	Population Social	4813.68
	Development System	4801.78
	Any Question	4759.28
	Fire Often	4734.89
	Natural Network	4722.66
	Build High	4707.14
	Serious Recognize	4523.10

PRODUCTS 17 x

- 10. Display product names and prices, sorted by price and then by name

SELECT NAME,PRICE FROM PRODUCTS ORDER BY PRICE desc , NAME asc;

```

2263 -- 10. Display product names and prices, sorted by price and then by name
2264 -- The merchandising or catalog team may want to list products from most expensive to cheapest. If
2265 -- multiple products have the same price, they should be sorted alphabetically for clarity on storefronts or
2266 -- printed catalogs
2267
2268 • SELECT NAME,PRICE FROM PRODUCTS ORDER BY PRICE desc , NAME asc;
2269

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	NAME	PRICE
▶	Response Indeed	4897.36
	Population Social	4813.68
	Development System	4801.78
	Any Question	4759.28
	Fire Often	4734.89
	Natural Network	4722.66
	Build High	4707.14
	Serious Recognize	4523.10

PRODUCTS 18 x

-- Level 2: Filtering and Formatting

-- 1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)

select * from orders where customer_id is null;


```

2273 -- 1. Retrieve orders where customer information is missing (possibly due to data migration or
2274 -- deletion)
2275 -- Used to identify orphaned orders or test data where customer_id is not linked.
2276
2277 • select * from orders where customer_id is null;
2278

```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
ORDER_ID	CUSTOMER_ID	ORDER_DATE	STATUS	TOTAL_AMOUNT
*	NULL	NULL	NULL	NULL

-- 2. Display customer names and emails using column aliases for frontend readability

```

SELECT NAME AS "CUSTOMER_NAMES" , EMAIL AS "CUSTOMER_EMAILS"
FROM CUSTOMERS ;

```

```

2279 -- 2. Display customer names and emails using column aliases for frontend readability
2280 -- Useful for feeding into frontend displays or report headings that require user-friendly labels
2281 |
2282 • SELECT NAME AS "CUSTOMER_NAMES" , EMAIL AS "CUSTOMER_EMAILS" FROM CUSTOMERS ;
2283

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
CUSTOMER_NAMES	CUSTOMER_EMAILS		
▶ Thomas Owens	user1@example.com		
Charles Grant	user2@example.com		
Kaitlin Richards	user3@example.com		
Christina Williams	user4@example.com		
David Allen	user5@example.com		
Mark Duke	user6@example.com		
Briana Wright	user7@example.com		
John Brvan	user8@example.com		

- 3. Calculate total value per item ordered by multiplying quantity and item price

```

SELECT ITEM_PRICE,QUANTITY,ITEM_PRICE* QUANTITY AS
'TOTAL_VALUE' FROM ORDER_ITEMS ;

```

```

2283
2284 -- 3. Calculate total value per item ordered by multiplying quantity and item price
2285 -- This can help generate per-line item bill details or invoice breakdowns
2286
2287 • SELECT ITEM_PRICE,QUANTITY,ITEM_PRICE* QUANTITY AS 'TOTAL_VALUE' FROM ORDER_ITEMS ;
2288

```

ITEM_PRICE	QUANTITY	TOTAL_VALUE
4707.14	2	9414.28
177.40	3	532.20
985.19	3	2955.57
2208.99	1	2208.99
4734.89	2	9469.78
4707.14	1	4707.14
4897.36	2	9794.72
4897.36	3	14692.08

-- 4. Combine customer name and phone number in a single column

```
SELECT CONCAT(NAME,"-",PHONE) AS CUSTOMER_DETAILS FROM CUSTOMERS;
```

```

2289 -- 4. Combine customer name and phone number in a single column
2290 -- Used to show brief customer summaries or contact lists
2291
2292 • SELECT CONCAT(NAME,"-----",PHONE) AS CUSTOMER_DETAILS FROM CUSTOMERS;
2293

```

CUSTOMER_DETAILS
Thomas Owens-----142-479-1945
Charles Grant-----9153947511
Kaitlin Richards-----2073473421
Christina Williams-----586-605-5061x06
David Allen----- (751)456-8289x1
Mark Duke----- (144)957-2811
Briana Wright-----223-833-9635
John Brvan-----045.568.0798x27

-- 5. Extract only the date part from order timestamps for date-wise reporting

```
SELECT DATE(ORDER_DATE) FROM ORDERS ;
```

```

2294 -- 5. Extract only the date part from order timestamps for date-wise reporting
2295 -- Helps group or filter orders by date without considering time.
2296
2297 • SELECT DATE(ORDER_DATE) FROM ORDERS ;
2298

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
DATE(ORDER_DATE)			
2025-03-02			
2024-10-09			
2025-05-08			
2024-09-19			
2025-04-08			
2024-10-25			
2024-07-29			
2024-07-30			

-- 6. List products that do not have any stock left

```

SELECT * FROM PRODUCTS WHERE STOCK_QUANTITY IS NULL OR
STOCK_QUANTITY =0;

```

```

2299 -- 6. List products that do not have any stock left
2300 -- This helps the inventory team identify out-of-stock items.
2301
2302 • SELECT * FROM PRODUCTS WHERE STOCK_QUANTITY IS NULL OR STOCK_QUANTITY =0;
2303
2304

```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	PRODUCT_ID	NAME	CATEGORY	PRICE	STOCK_QUANTITY	ADDED_ON
*	NULL	NULL	NULL	NULL	NULL	NULL

-- Level 3: Aggregations

-- 1. Count the total number of orders placed

```

SELECT COUNT(*) AS TOTAL_ORDERS FROM ORDERS;

```



```

2306
2307 -- 1. Count the total number of orders placed
2308 -- Used by business managers to track order volume over time.
2309
2310 • SELECT COUNT(*) AS TOTAL_ORDERS FROM ORDERS;
2311

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	TOTAL_ORDERS				
	400				

-- 2. Calculate the total revenue collected from all orders

```
SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM ORDERS;
```

```

2313 -- 2. Calculate the total revenue collected from all orders
2314 -- This gives the overall sales value.
2315
2316 • SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM ORDERS;
2317

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	TOTAL_REVENUE				
	6960973.66				

-- 3. Calculate the average order value

```
SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE FROM ORDERS;
```

```

2318      -- 3. Calculate the average order value
2319      -- Used for understanding customer spending patterns.
2320
2321 •      SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE FROM ORDERS;
2322

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
	AVERAGE_ORDER_VALUE		
	17402.434150		

-- 4. Count the number of customers who have placed at least one order

```
select count(distinct customer_id) as 'number_of_customers' from orders;
```

```

2323      -- 4. Count the number of customers who have placed at least one order
2324      -- This identifies active customers.
2325
2326 •      SELECT COUNT(DISTINCT CUSTOMER_ID) AS 'NUMBER_OF_CUSTOMERS' FROM ORDERS;
2327

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
	NUMBER_OF_CUSTOMERS		
	30		

-- 5. Find the number of orders placed by each customer

```
SELECT CUSTOMER_ID,COUNT(ORDER_ID) AS 'NUMBERS_OF_ORDERS'
FROM ORDERS GROUP BY CUSTOMER_ID ;
```

```

2328      -- 5. Find the number of orders placed by each customer
2329      -- Helpful for identifying top or repeat customers.
2330
2331 •      SELECT CUSTOMER_ID,COUNT(ORDER_ID) AS 'NUMBERS_OF_ORDERS' FROM ORDERS GROUP BY CUSTOMER_ID ;
2332

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
	CUSTOMER_ID	NUMBERS_OF_ORDERS	
	1	12	
	2	17	
	3	17	
	4	9	
	5	14	
	6	16	
	7	13	
	8	10	

Result 5 x

Output

-- 6. Find total sales amount made by each customer

```
select customer_id,sum(total_amount) as "total_sales_amount" from orders group by customer_id;
```

```
2333 -- 6. Find total sales amount made by each customer
2334
2335 • select customer_id,sum(total_amount) as "total_sales_amount" from orders group by customer_id;
2336
```

customer_id	total_sales_amount
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78

Result 6 x

-- 7. List the number of products sold per category

```
SELECT p.CATEGORY,SUM(O.QUANTITY) AS
"NUMBER_OF_PRODUCTS_SOLD" FROM ORDER_ITEMS o JOIN PRODUCTS p
ON o.PRODUCT_ID = p.PRODUCT_ID GROUP BY p.CATEGORY;
```

```
2337 -- 7. List the number of products sold per category
2338 -- This helps category managers assess performance by department.
2339
2340 • SELECT p.CATEGORY,SUM(O.QUANTITY) AS "NUMBER_OF_PRODUCTS_SOLD" FROM ORDER_ITEMS o JOIN PRODUCTS p ON o.PRODUCT_ID = p.PRODUCT_ID GROUP BY p.CATEGORY;
2341
```

CATEGORY	NUMBER_OF_PRODUCTS_SOLD
Home	443
Clothing	559
Electronics	687
Toys	405
Books	350

Result 9 x

-- 8. Find the average item price per category

```
SELECT P.CATEGORY,SUM(O.ITEM_PRICE * O.QUANTITY) /
SUM(O.QUANTITY) AS "AVERAGE_ITEM_PRICE" FROM ORDER_ITEMS O JOIN
PRODUCTS P ON O.PRODUCT_ID=P.PRODUCT_ID GROUP BY P.CATEGORY;
```

```
2342 -- 8. Find the average item price per category
2343 -- Useful to compare pricing across departments.
2344
2345 • SELECT P.CATEGORY,SUM(O.ITEM_PRICE * O.QUANTITY) / SUM(O.QUANTITY) AS "AVERAGE_ITEM_PRICE"
2346 FROM ORDER_ITEMS O JOIN PRODUCTS P ON O.PRODUCT_ID=P.PRODUCT_ID GROUP BY P.CATEGORY;
2347
```

CATEGORY	AVERAGE_ITEM_PRICE
Home	2401.922709
Clothing	3466.202898
Electronics	2711.882838
Toys	2412.987432
Books	3197.117314

- 9. Show number of orders placed per day

```
SELECT DATE (ORDER_DATE) AS ORDER_DATE,COUNT(*) AS  
"NUMBER_OF_ORDERS" FROM ORDERS GROUP BY DATE(ORDER_DATE);
```

```
2348 -- 9. Show number of orders placed per day  
2349 -- Used to track daily business activity and demand trends.  
2350  
2351 • SELECT DATE (ORDER_DATE) AS ORDER_DATE,COUNT(*) AS "NUMBER_OF_ORDERS" FROM ORDERS GROUP BY DATE(ORDER_DATE);  
2352
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	ORDER_DATE	NUMBER_OF_ORDERS
▶	2025-03-02	3
	2024-10-09	2
	2025-05-08	2
	2024-09-19	3
	2025-04-08	3
	2024-10-25	3
	2024-07-29	2
	2024-07-30	2

Result 11 x

-- 10. List total payments received per payment method

```
SELECT P.METHOD ,SUM(P.AMOUNT_PAID) AS "TOTAL_PAYMENTS" FROM  
PAYMENTS P GROUP BY P.METHOD;
```

```
2353 -- 10. List total payments received per payment method  
2354 -- Helps the finance team understand preferred transaction modes.  
2355  
2356 • SELECT P.METHOD ,SUM(P.AMOUNT_PAID) AS "TOTAL_PAYMENTS" FROM PAYMENTS P GROUP BY P.METHOD;  
2357  
2358
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	METHOD	TOTAL_PAYMENTS
▶	Credit Card	1754603.10
	Net Banking	1658383.90
	UPI	1617408.78
	Debit Card	1930577.88

-- Level 4: Multi-Table Queries (JOINS)

-- 1. Retrieve order details along with the customer name (INNER JOIN)

```
SELECT C.NAME AS  
"CUSTOMER_NAME",O.ORDER_ID,O.ORDER_DATE,O.TOTAL_AMOUNT,O.ST  
ATUS FROM ORDERS O INNER JOIN CUSTOMERS C ON C.CUSTOMER_ID =  
O.CUSTOMER_ID;
```

```

2359 -- Level 4: Multi-Table Queries (JOINS)
2360
2361 -- 1. Retrieve order details along with the customer name (INNER JOIN)
2362 -- Used for displaying which customer placed each order.
2363
2364 • SELECT C.NAME AS "CUSTOMER_NAME",O.ORDER_ID,O.ORDER_DATE,O.TOTAL_AMOUNT,O.STATUS FROM ORDERS O INNER JOIN CUSTOMERS C ON C.CUSTOMER_ID = O.CUSTOMER_ID;
2365

```

	CUSTOMER_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	STATUS
▶	Thomas Owens	14	2024-09-24 21:21:38	15803.34	Shipped
	Thomas Owens	17	2024-08-19 21:17:57	11173.77	Pending
	Thomas Owens	61	2024-12-26 23:21:58	13053.00	Shipped
	Thomas Owens	76	2025-01-14 22:59:54	23506.81	Pending
	Thomas Owens	92	2024-09-26 11:33:58	834.75	Shipped
	Thomas Owens	109	2025-03-17 04:56:18	12190.14	Shipped
	Thomas Owens	127	2025-06-10 18:01:32	20160.64	Pending
	Thomas Owens	135	2025-03-11 22:58:23	8891.79	Pending

Result 15 x

- 2. Get list of products that have been sold (INNER JOIN with order items)

```

2366 -- 2. Get list of products that have been sold (INNER JOIN with order_items)
2367 -- Used to find which products were actually included in orders.
2368
2369 • SELECT DISTINCT oi.product_id, o.order_id FROM orders o INNER JOIN order_items oi ON o.order_id = oi.order_id;
2370

```

	product_id	order_id
▶	16	14
	8	14
	32	17
	37	17
	8	61
	6	61
	30	76
	6	76

Result 16 x

Output

- 3. List all orders with their payment method (INNER JOIN)

```

2371 -- 3. List all orders with their payment method (INNER JOIN)
2372 -- Used by finance or audit teams to see how each order was paid for
2373
2374 • select o.order_id, p.method from orders o inner join payments p on o.order_id= p.order_id;
2375

```

	order_id	method
▶	14	UPI
	17	UPI
	61	Net Banking
	76	Credit Card
	92	Net Banking
	109	UPI
	127	Debit Card
	135	Debit Card

Result 17 x

Output

-- 4. Get list of customers and their orders (LEFT JOIN)

```
2376 -- 4. Get list of customers and their orders (LEFT JOIN)
2377 -- Used to find all customers and see who has or hasn't placed orders.
2378
2379 • select c.name as "customer_name", o.order_id from customers c left join orders o on c.customer_id = o.customer_id;
2380
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	customer_name	order_id
▶	Thomas Owens	14
	Thomas Owens	17
	Thomas Owens	61
	Thomas Owens	76
	Thomas Owens	92
	Thomas Owens	109
	Thomas Owens	127
	Thomas Owens	135

Result 18 x

5. List all products along with order item quantity (LEFT JOIN)

```
2381 -- 5. List all products along with order item quantity (LEFT JOIN)
2382 -- Useful for inventory teams to track what sold and what hasn't.
2383
2384 • select p.name as "product_name", oi.quantity from products p left join order_items oi on p.product_id = oi.product_id;
2385
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	product_name	quantity
▶	Plant No	1
	Plant No	3
	Plant No	1
	Plant No	1
	Plant No	2
	Plant No	3
	Plant No	2
	Plant No	2

Result 19 x

-- 6. List all payments including those with no matching orders (RIGHT JOIN)

```
select p.method,o.order_id from orders o right join payments p on o.order_id =
p.order_id;
```

```
2386 -- 6. List all payments including those with no matching orders (RIGHT JOIN)
2387 -- Rare but used when ensuring all payments are mapped correctly.
2388
2389 • select p.method,o.order_id from orders o right join payments p on o.order_id = p.order_id;
2390
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	method	order_id
▶	Credit Card	1
	Net Banking	2
	Credit Card	3
	UPI	4
	UPI	5
	UPI	6
	UPI	7
	Net Banking	8

Result 21 x

-- 7. Combine data from three tables: customer, order, and payment

select c.customer_id, c.name,c.email,c.phone,c.created_at,o.order_id,p.method from customers c inner join orders o on c.customer_id = o.customer_id inner join payments p on o.order_id= p.order_id;

```
2391 -- 7. Combine data from three tables: customer, order, and payment
2392 -- Used for detailed transaction reports.
2393
2394 • select c.customer_id, c.name,c.email,c.phone,c.created_at,o.order_id,p.method from customers c
2395 inner join orders o on c.customer_id = o.customer_id inner join payments p on o.order_id= p.order_id;
2396
```

customer_id	name	email	phone	created_at	order_id	method
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	14	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	17	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	61	Net Banking
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	76	Credit Card
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	92	Net Banking
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	109	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	127	Debit Card
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	135	Debit Card

- Level 5: Subqueries (Inner Queries)

-- 1. List all products priced above the average product price

select * from products;

select * from products where price >(select avg(price) from products);

```

2398
2399 -- 1. List all products priced above the average product price
2400 -- Used by pricing analysts to identify premium-priced products.
2401 • select * from products;
2402 • select * from products where price > (select avg(price) from products);
2403
2404 -- 2. Find customers who have placed at least one order

```

PRODUCT_ID	NAME	CATEGORY	PRICE	STOCK_QUANTITY	ADDED_ON
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53

-- 2. Find customers who have placed at least one order

```

select c.name,o.order_id from orders o inner join customers c on
o.customer_id=c.customer_id;

```

```

2404 -- 2. Find customers who have placed at least one order
2405 -- Used to identify active customers for loyalty campaigns.
2406
2407 • select c.name,o.order_id from orders o inner join customers c on o.customer_id=c.customer_id;
2408

```

name	order_id
Thomas Owens	14
Thomas Owens	17
Thomas Owens	61
Thomas Owens	76
Thomas Owens	92
Thomas Owens	109
Thomas Owens	127
Thomas Owens	135

-- 3. Show orders whose total amount is above the average for that customer

```

select * from orders o where total_amount > (select avg(o2.total_amount) from
orders o2 where o2.customer_id = o.customer_id);

```

```

2409 -- 3. Show orders whose total amount is above the average for that customer
2410 -- Used to detect unusually high purchases per customer
2411
2412 • select * from orders o where total_amount > (select avg(o2.total_amount) from orders o2 where o2.customer_id = o.customer_id);
2413

```

ORDER_ID	CUSTOMER_ID	ORDER_DATE	STATUS	TOTAL_AMOUNT
6	29	2024-10-25 07:33:59	Cancelled	31050.17
8	19	2024-07-30 22:49:49	Cancelled	32714.06
9	6	2025-06-10 17:00:25	Pending	24219.20
10	28	2025-02-16 12:45:59	Delivered	24342.52
14	1	2024-09-24 21:21:38	Shipped	15803.34
16	5	2024-09-09 03:00:40	Shipped	22856.73
18	13	2024-06-15 12:57:04	Cancelled	32001.24
21	22	2025-03-12 18:53:46	Pending	25364.11

-- 4. Display customers who haven't placed any orders

SELECT * FROM customers c WHERE c.customer_id NOT IN (SELECT customer_id FROM orders);

```

2414 -- 4. Display customers who haven't placed any orders
2415 -- Used for re-engagement campaigns targeting inactive users.
2416
2417 • SELECT * FROM customers c WHERE c.customer_id NOT IN (SELECT customer_id FROM orders );
2418

```

CUSTOMER_ID	NAME	EMAIL	PHONE	CREATED_AT
NULL	NULL	NULL	NULL	NULL

5. Show products that were never ordered

```

2419 -- 5. Show products that were never ordered
2420 -- Helps with inventory clearance decisions or product deactivation.
2421 • SELECT p.product_id, p.name FROM products p LEFT JOIN order_items oi ON p.product_id = oi.product_id WHERE oi.product_id IS NULL;
2422

```

product_id	name
------------	------

- 6. Show highest value order per customer

select * from orders o where total_amount = (SELECT MAX(o2.total_amount) FROM orders o2 where o2.customer_id=o.customer_id);

```

2423 -- 6. Show highest value order per customer
2424 -- Used to identify the largest transaction made by each customer.
2425
2426 • select * from orders o where total_amount = (SELECT MAX(o2.total_amount) FROM orders o2 where o2.customer_id=o.customer_id);
2427

```

ORDER_ID	CUSTOMER_ID	ORDER_DATE	STATUS	TOTAL_AMOUNT
18	13	2024-06-15 12:57:04	Cancelled	32001.24
33	8	2025-04-27 01:26:11	Pending	36147.09
62	10	2025-06-13 23:21:01	Cancelled	35723.17
66	27	2024-06-19 09:41:16	Pending	26176.05
68	24	2025-03-15 07:27:20	Delivered	36159.92
80	12	2025-03-28 18:42:12	Delivered	38656.26
107	14	2025-02-01 02:02:18	Shipped	48042.06
121	16	2024-11-21 16:50:01	Pending	37415.67

-- 7. Highest Order Per Customer (Including Names)

select c.customer_id,c.name as "customer_name",max(o.total_amount) as highest_order from orders o inner join customers c on o.customer_id=c.customer_id group by c.customer_id, c.name;

```

2428 -- 7. Highest Order Per Customer (Including Names)
2429 -- Used to identify the largest transaction made by each customer. Outputs name as well.
2430
2431 • select c.customer_id,c.name as "customer_name",max(o.total_amount) as highest_order
2432 from orders o inner join customers c on o.customer_id=c.customer_id group by c.customer_id, c.name;
2433

```

customer_id	customer_name	highest_order
1	Thomas Owens	32015.16
2	Charles Grant	42056.04
3	Kaitlin Richards	41679.11
4	Christina Williams	25747.34
5	David Allen	39921.78
6	Mark Duke	39003.19
7	Briana Wright	28589.04
8	John Brvan	36147.09

-- Level 6: Set Operations

-- 1. List all customers who have either placed an order or written a product review

SELECT c.customer_id, c.name FROM customers c WHERE c.customer_id IN (SELECT customer_id FROM orders) OR c.customer_id IN (SELECT customer_id FROM product_reviews);

```

2434 -- Level 6: Set Operations
2435
2436 -- 1. List all customers who have either placed an order or written a product review
2437 -- Used to identify engaged customers from different activity areas.
2438
2439 • SELECT c.customer_id, c.name FROM customers c WHERE c.customer_id IN (SELECT customer_id FROM orders)
2440 OR c.customer_id IN (SELECT customer_id FROM product_reviews);
2441

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	name
1	Thomas Owens
2	Charles Grant
3	Kaitlin Richards
4	Christina Williams
5	David Allen
6	Mark Duke
7	Briana Wright
8	John Brvan

customers 37 x

2. List all customers who have placed an order as well as reviewed a product [intersect not supported]

```

SELECT c.customer_id, c.name FROM customers c WHERE c.customer_id IN
(SELECT customer_id FROM orders) AND c.customer_id IN (SELECT customer_id
FROM product_reviews);

```

```

2441
2442 -- 2. List all customers who have placed an order as well as reviewed a product [intersect not
2443 -- supported]
2444 -- Used to identify highly engaged customers for rewards.
2445
2446 • SELECT c.customer_id, c.name FROM customers c WHERE c.customer_id IN (SELECT customer_id FROM orders)
2447 AND c.customer_id IN (SELECT customer_id FROM product_reviews);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	name
1	Thomas Owens
2	Charles Grant
4	Christina Williams
6	Mark Duke
7	Briana Wright
9	Jason Thompson
10	Shawn Hill
11	Walter Jenkins

customers 30 x